

The portable Foundry, drawn as it is built now

DRAWING **FND-02**

REV 2026-08-02

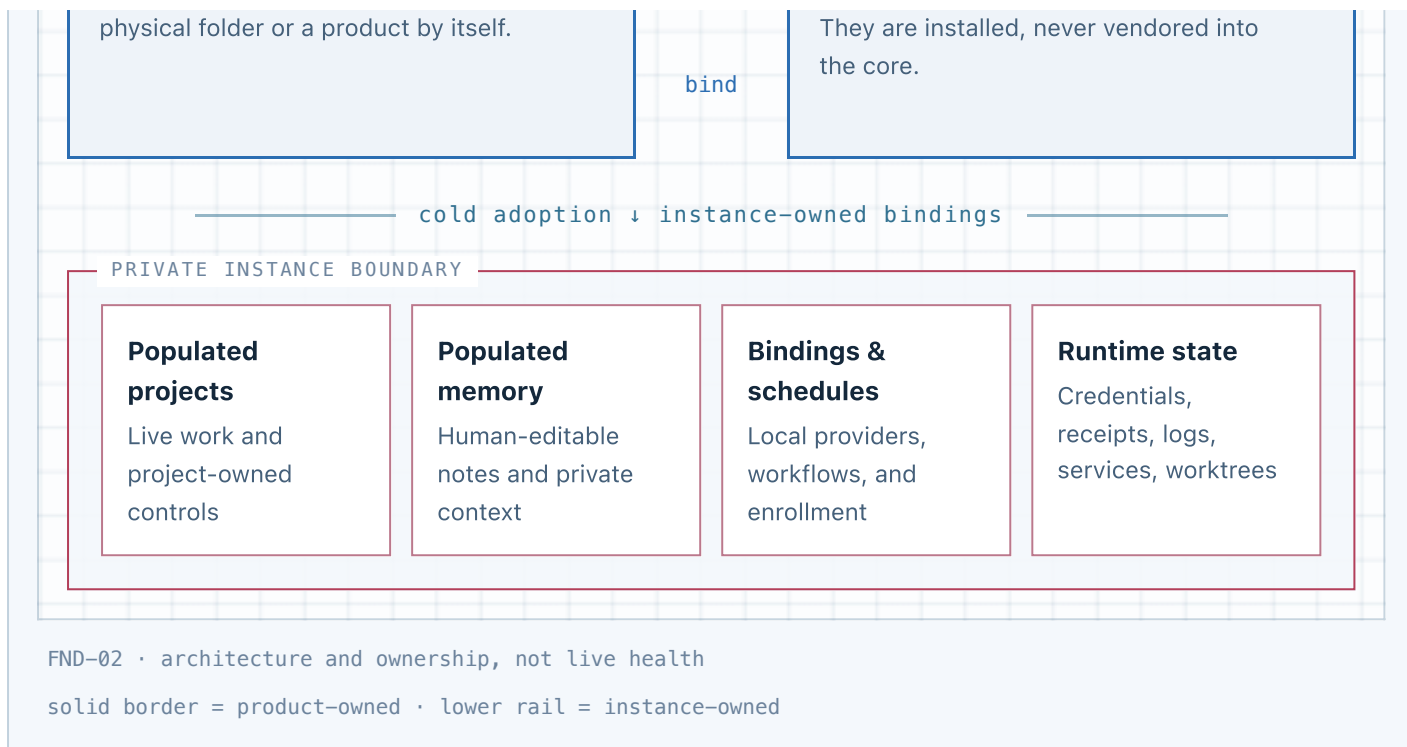
4 native Halls

1 product · many instances

FND is simply the identifier for the Foundry drawing series. FND-01 preserves the original two-building concept; this successor shows what consolidation changed: one portable core, clean ownership boundaries, and an independently checked route from source to adoption.

One product core, adopted into a private instance





— PLATE 2 — THE HALLS

The old “Sockets” wing became four native Halls

■ Forge · makes

The production floor. It owns development, evaluation, packaging, release mechanics, and the reusable harness—not project truth.

NATIVE TRACKED SOURCE

■ Assay · judges

The independent inspection bench. It reads evidence and returns a decision; it does not silently repair the thing it is judging.

READ-ONLY JUDGMENT

■ Ward · connects

The integration clinic. It verifies that optional Modules meet their socket contracts and reports transport or health failures.

CONTRACT AND HEALTH BOUNDARY

■ Gatehouse · contains

The authority checkpoint. It is the home for Job Orders and containment enforcement; the Hall exists while the fuller capability remains scaffolded.

NATIVE HALL · CAPABILITY SCAFFOLDED

A public product is built, assayed, then adopted

01 · SOURCE

Freeze a producer commit

The build begins from one immutable revision, not a mutable working directory.

02 · FORGE

Construct the clean tree

Declared paths enter; installed products, instance data, and generated state stay out.

03 · ASSAY

Judge the exact artifact

A separate read-only pass binds its decision to the immutable tree digest.

04 · INTEGRATION

Publish the accepted product

Automation may deliver to staging. Production promotion remains an owner decision.

05 · ADOPT

Bring up a fresh instance

Native Halls verify in place; optional Modules install; private data remains local.

What materially changed

CONCERN	FND-01 CONCEPT	FND-02 CURRENT MODEL
Shape	Two host-specific buildings were the main picture.	One portable product can be adopted by many private instances and hosts.
Native structure	Helpers appeared as wires around the buildings.	Forge, Assay, Ward, and Gatehouse are native tracked Halls that arrive with the clone.
Sockets	"Socket" doubled as a helper category and a physical wing.	A socket is only a stable capability contract housed inside a Hall.
Modules	Named helpers looked bolted directly to a host.	Optional Modules remain independent products, installed by manifest and reached only through contracts.
Projects & memory	Shown mainly as one shared host vault.	The product ships generic Projects/Wiki capabilities; populated work and memory belong to the instance.
Reusable work system	Crews were present, but their ownership was mostly conceptual.	Roles, Captain scheduling, and deterministic tools are explicit portable core.
Release proof	The route from authoring source to product was not drawn.	An immutable Forge artifact receives independent Assay judgment before staging and cold adoption.

The light levels survived, with one careful addition

L0 · sleeping No model Deterministic checks exit when nothing relevant changed. IMPLEMENTED
L0.5 · sweep Micro-wake A self-extinguishing deterministic check returns to L0 or wakes L1. APPROVED · CODE PENDING
L1 · dispatcher Captain Reads grounded state, prioritizes, and decides whether real work exists. IMPLEMENTED
L2 · workers Build and audit One claimed vertical slice is implemented and independently checked. OPERATING MODEL
L3 · executive Escalation Reserved for ambiguity, novel architecture, and major conflicts. EXCEPTION ONLY

What am I looking at?

FND-01 drew the Foundry as two factories with doors and helpers on wires. That was useful: it made the idea tangible. FND-02 keeps the factory metaphor but draws the ownership lines we have since built.

The **factory itself is now a portable kit**. Its four permanent rooms are the Forge, Assay, Ward, and Gatehouse. The kit also carries the crew contracts, scheduling mechanics, project and memory scaffolding, and deterministic tools needed to start cleanly somewhere else.

Optional helpers are still separate. They plug in through **socket contracts**; they are not hidden inside the walls. A deployed instance owns its actual projects, memory, credentials, bindings, schedules, and runtime evidence, so none of that private material leaks into the reusable product.

Most importantly, the public kit is no longer copied by hand. The Forge builds one exact artifact, the Assay independently checks that exact artifact, staging receives only the accepted tree, and a cold adoption proves it can wake up somewhere fresh.